



PEAK: a Light-Weight Profiler for HPC Systems

Yinzhi Wang
iwang@tacc.utexas.edu

Texas Advanced Computing Center
Austin, Texas, USA

Junjie Li

jli@tacc.utexas.edu

Texas Advanced Computing Center
Austin, Texas, USA

ABSTRACT

In the context of the expanding landscape of contemporary High-Performance Computing (HPC) applications from petascale to exascale, the pursuit of performance optimization emerges as a significant impediment within software development endeavors. In the meantime, the escalating intricacies inherent in parallel architectures and systems serve to compound the challenges associated with performance enhancement. Here, we introduce PEAK (Performance Evaluation and Analysis Kit), a light-weight profiling tool developed with a specific focus on large-scale HPC applications. Using Dynamic Binary Instrumentation, PEAK is able to profile large-scale multi-threaded, multi-process applications with low overhead and high accuracy. We analyzed the overhead and accuracy of PEAK using synthetic benchmarks and real applications and compared it against the other widely used HPC profiling tools available. Our demonstration underscores that PEAK exhibits comparable overhead and accuracy to alternative profiling tools, while preserving its inherent simplicity.

CCS CONCEPTS

• **Software and its engineering** → **Software performance**; • **General and reference** → **Performance**.

KEYWORDS

profiling, application performance, system tools

ACM Reference Format:

Yinzhi Wang and Junjie Li. 2023. PEAK: a Light-Weight Profiler for HPC Systems. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023)*, November 12–17, 2023, Denver, CO, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3624062.3624143>

1 INTRODUCTION

In the realm of modern scientific computing, where large-scale simulations, big data analysis, and complex modeling are the norm, High-Performance Computing (HPC) has emerged as a catalyst to scientific discovery. As the computational requirements of communities surge from the petascale to the exascale, and with Moore's Law diminishing in impact, the pursuit of extracting optimal performance from HPC systems has taken center stage. Amidst this pursuit, profiling tools have risen to prominence as indispensable

instruments that unveil the intricate interplay between software and hardware within the HPC ecosystem.

HPC profiling tools provide a detailed and insightful view of how applications perform on parallel architectures. These tools go beyond simple observation, exposing inefficiencies, bottlenecks, and hidden performance issues that can hinder the efficient use of computational resources. By thoroughly analyzing program execution, memory allocation, communication patterns, and parallelism, profiling tools empower not just HPC code developers but also scientists and engineers, enabling them to optimize their code with precision and driving advancements in scientific discovery.

Here, we introduce a novel, streamlined profiler named PEAK (Performance Evaluation and Analysis Kit) meticulously crafted to cater to the requirements of prevalent HPC applications. PEAK showcases a lightweight design alongside an intuitive user interface, ensuring an enhanced profiling experience for HPC users. Employing Dynamic Binary Instrumentation, PEAK establishes itself as a seamless, plug-and-play instrument, circumventing the necessity for additional compiler flags or options. It is also built to support MPI and pthread so it can profile intricate, large-scale multi-threaded and multi-process applications, without any onerous user intervention. PEAK attaches to the profiled application through the utilization of LD_PRELOAD with the PEAK library and it interacts with users through intuitive environment variables and concise command-line output. In this paper, we will elaborate on the design of PEAK, and then we use multiple test cases comparing with a few other popular profilers to evaluate overhead and accuracy.

2 BACKGROUND AND RELATED WORK

2.1 Dynamic Binary Instrumentation

Dynamic Binary Instrumentation (DBI) is a powerful technique used in computer science and software engineering to analyze, profile, and manipulate the execution of binary code at runtime. It involves inserting instrumentation code into a compiled binary program to monitor and control its behavior as it runs, without requiring access to the original source code. This technique is particularly useful for gaining insights into the runtime behavior of software, identifying performance bottlenecks, optimizing code, and conducting various forms of analysis.

DBI is based on the technique called trampoline, which consists of specialized code segments that redirect the execution flow to designated instrumentation routines. Intercepting a function with a trampoline involves allocating memory for trampoline code that holds redirection instructions. The target function's initial bytes are then modified to jump to the trampoline code. When the function is invoked, this jump redirects to the trampoline, enabling invocation of the instrumentation routine and context management. Subsequently, the trampoline bounces back to the original flow of execution. This process is illustrated in Figure 1.



This work is licensed under a Creative Commons Attribution International 4.0 License.

SC-W 2023, November 12–17, 2023, Denver, CO, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0785-8/23/11.

<https://doi.org/10.1145/3624062.3624143>

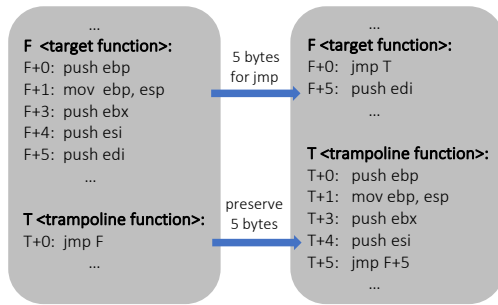


Figure 1: Trampoline and Target Functions before and after Interception. This figure is adapted from [4], showcasing x86 instructions as an illustrative example. It is worth noting that other instruction sets also feature their own variations of trampolines.

There are several DBI tools available that enable the analysis, profiling, and manipulation of binary code at runtime (e.g., Dyninst[2], DynamoRIO[3], PIN[6]). Every individual tool possesses distinct features and capabilities, each employing a more intricate trampoline implementation compared to the simplistic representation depicted in Figure 1. In particular, Dyninst holds the foremost position among profilers that utilize DBI due to its low overhead, effective maintenance, and rich feature set. However, the rich feature set comes with the drawback of a lengthy list of dependencies, posing challenges for deploying the library, particularly in the HPC environment, where users lack root access.

In PEAK, we use the foundational C library of Frida [8]. It is a cross-platform DBI library that supports both x86 and Arm architectures. Given its primary focus on instrumenting applications across mobile devices, we observed a slightly lower overhead with Frida compared to Dyninst. Furthermore, Frida offers its release through a devkit, encompassing solely a header file and a static library with no additional dependency. This streamlined packaging greatly facilitates the library’s deployment on the HPC system.

2.2 Profiling Tools on HPC

There are several profiling tools commonly used on HPC systems to analyze and optimize program performance. These tools cover a range of profiling techniques and offer insights into different aspects of an application’s behavior. An important distinction among profiling tools is whether they use sampling or instrumentation techniques to gather performance data.

Sampling profilers collect performance data by periodically interrupting the program’s execution and recording the state of the program at that moment. This interruption is often triggered by a timer or an external event. Sampling does not require modifying the program’s source code, making it relatively non-intrusive. However, it might not capture all events or provide fine-grained details. Profilers that use sampling include HPCtoolkit[1] and MAP[5].

Instrumentation involves modifying the program’s source code or binary to insert measurement points that collect performance data during execution. This approach offers detailed and accurate insights into the program’s behavior but can introduce some overhead

due to the added instrumentation code. Profilers employing instrumentation encompass TAU[10] and Score-P[7]. Notably, there exist profilers that combine both sampling and instrumentation techniques to serve distinct functionalities, as exemplified by VTune[9].

Although well-equipped with a plethora of advanced features, the established profiling tools bear a resemblance to the comprehensive 20-volume Oxford English Dictionary. These tools prove invaluable for seasoned HPC developers, yet their utilization demands a significant learning curve and intricate operational complexities. Furthermore, certain potent performance tools, such as TAU, HPCtoolkit, and MAP, lack a command-line interface for viewing profiling results, making them unfriendly to a typical HPC workflow or routine performance assessment.

On the flip side, PEAK aims to stand as a convenient pocket dictionary, offering support for fundamental yet vital performance needs through a user-friendly setup that accommodates both experienced HPC developers and everyday users. We implemented PEAK with DBI to enable selective instrumentation of user-specified functions, ensuring minimal overhead without compromising accuracy.

3 IMPLEMENTATION

The PEAK library attaches to user applications through LD_PRELOAD such that the library and the application shares the same virtual memory. It places two function pointers into the `.init_array` and `.fini_array` sections, respectively. These sections, specific to the ELF (Executable and Linkable Format) binary format primarily used on Linux and Unix-like systems, store pointers to functions that need to be executed during program initialization and termination. During the initialization phase, PEAK conducts a search for symbols marked for instrumentation. It subsequently attaches instrumentation code segments into the entry points and return addresses of the respective functions. In the termination phase, PEAK gathers instrumented data and removes the code segments to reinstate the original memory content of the application. This process is illustrated in Figure 2.

An inherent challenge encountered by profilers utilizing instrumentation is the potential for unbounded overhead, particularly when profiling small functions that undergo frequent invocation within the application. This predicament is notably pronounced when the cost of the instrumentation code approximates or surpasses that of the function being instrumented. PEAK addresses this issue through a preliminary implementation of a runtime detachment mechanism, which disengages the instrumentation code segment when the aggregated overhead surpasses a user-defined time threshold. User can explicitly control the trade-off between profiling overhead and accuracy by configuring the threshold. It’s important to note that a comparable trade-off exists within sampling profilers, where users manipulate the sampling frequency. However, it’s crucial to acknowledge that accuracy in sampling profilers is inherently bounded, as the sampling interval cannot be reduced to zero.

Designed as a profiling tool for HPC applications, PEAK provides support for both multi-threading and multi-processing environments. The multi-threading support of PEAK is implemented by tracking the `pthread_create` functions and keeping the instrumented data distinct for individual threads. Given that `pthread`

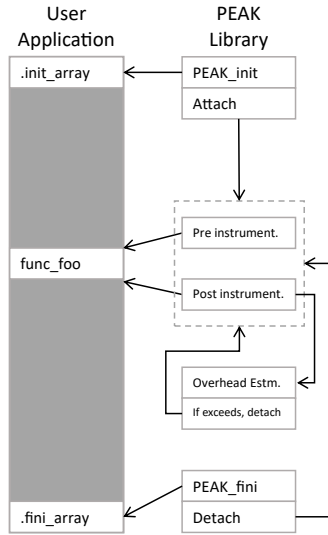


Figure 2: Illustration of PEAK Library Instrumentation Process for Function 'foo' in a User Application. The library is attached to the application process through LD_PRELOAD. The configuration and removal of the instrumentation code occur within the `.init_array` and `.fini_array` sections, respectively. The instrumentation code will also be detached if it surpasses a user-defined overhead limit.

serves as the foundational threading implementation for a multitude of threading libraries, including OpenMP, commonly employed within HPC applications, PEAK's efficacy spans a wide spectrum. The modular design of PEAK allows for the potential integration of other threading implementations, should the need arise. The multi-processing support of PEAK mainly aims for the Message Passing Interface (MPI) library. It uses the environment variables (i.e., `PMI_RANK`, `MV2_COMM_WORLD_RANK`, `OMPI_COMM_WORLD_RANK`) to discern whether the process stems from the MPI launcher, which should cover all major MPI distributions. When the MPI environment is detected, PEAK's termination phase executes an additional sequence of `MPI_Reduce` calls. This serves the purpose of amalgamating instrumentation data, thereby yielding a comprehensive summary across the distributed MPI processes.

At present, PEAK focuses on capturing fundamental metrics for each instrumented function, encompassing call count, inclusive time, and exclusive time. It also offers straightforward statistical insights, including per-thread and per-MPI rank averages. Notably, our ongoing efforts are directed towards expanding PEAK's measurement capabilities, with the intention of incorporating additional metrics in the near future.

4 EVALUATION

4.1 Overview

In assessing PEAK's effectiveness, we undertake a comparative analysis with several well-established profiling tools commonly used by the HPC community, including TAU (2.28.2), Arm MAP

(20.0.2), mpiP (3.5), and Intel APS – Application Performance Snapshot (2023.1.0). Our primary focus centers on evaluating PEAK's overhead and accuracy.

4.2 Profiling Overhead

In order to comprehend the profiling overhead, we devised two tiny programs. One program invokes an empty function, while the other calls a `nanosleep` function, both executed 10,000,000 times. Then, we assess the per-call overhead by dividing the time prolonged by the profiler by the total number of calls.

Table 1: Profiling Overhead Per Function Call[†]

Test Case	TAU	PEAK	PEAK [*]
empty function	$3.548 \pm 0.029\mu s$	$0.358 \pm 0.002\mu s$	$4.873 \pm 0.050\mu s$
nanosleep	$4.763 \pm 0.083\mu s$	$0.626 \pm 0.034\mu s$	$5.174 \pm 0.022\mu s$

[†] statistics out of 10 runs, each run with 10 million function calls.

^{*} through detachment enabled code path, but no profiling target is detached.

Table 1 presents a comparison between PEAK and TAU, with TAU being selected as the benchmark due to its standing as a prominent instrumented profiler for HPC. Note that PEAK's overhead closely approximates that of TAU's. Considering this version of TAU employs recompilation for the insertion of its instrumentation code – a process inherently inducing less overhead than the use of DBI – PEAK's current overhead is commendable. Due to the complexity within the code path enabled for detachment, we also find that the overhead of PEAK can be reduced by an order of magnitude when the detachment feature is completely disabled.

4.3 Profiling Accuracy

We use the widely known lattice-QCD code MILC as our test application. The gauge grid is set to $32 \times 32 \times 32 \times 32$, lattice spacing is 0.03 fm. The strange and charm quark masses are set to their physical values, while the up and down quarks are set to their average values. This particular test case runs in single precision, with a step size of 0.0125 and 40 steps in total. For all MILC test jobs in this paper, we run MILC in MPI mode and use 4 nodes on Frontera[11] with 32 MPI ranks per node.

To assess the profiling accuracy, we first compare PEAK's result with those from two extensively used MPI profilers: Intel APS and mpiP[12]. Note that in practice, MPI profiling interface is preferred instead. Table 2 presents the five most resource-intensive MPI functions. By comparing the percentage of time spent in each function relative to the total runtime, it is evident that PEAK's measurements closely align with the other two profilers. Note that PEAK's measurement is consistently higher when instrumenting smaller functions. This is due to the inherent overhead stemming from the trampoline implementation, which introduces extra costs from additional processes such as register backup and restoration. It is important to acknowledge that Intel APS and mpiP, being specialized MPI profilers, integrate into the application through the MPI profiling interface, yielding significantly lower overheads. Nevertheless, PEAK demonstrates its capability to deliver accurate measurements while maintaining a comparatively modest overhead.

4.4 Application Profiling

We use the same MILC test case for a comprehensive application profiling comparison. Within this context, PEAK is configured to

Table 2: Average time for top 5 MPI functions in MILC

Function	Intel APS	mpiP	PEAK *
MPI_Wait:	40.12s (9.07%)	41.72s (9.36%)	42.12 (9.28%)
MPI_Isend:	7.95s (1.80%)	8.13s (1.82%)	8.36 (1.84%)
MPI_Allreduce:	4.82s (1.09%)	5.09s (1.14%)	7.50 (1.65%)
MPI_Init_thread:	1.14s (0.26%)	N/A †	1.45 (0.32%)
MPI_Irecv:	0.71s (0.16%)	0.75s (0.17%)	0.97 (0.21%)
Total MPI time: ‡	54.83s (12.40%)	55.65s (12.49%)	60.76s (13.40%)
Total App Time:	442.21s (100%)	445.47s (100%)	453.68s (100%)

* with detachment feature turned off.

† mpiP did not capture this function.

‡ The total number of MPI function calls in this test case is over 400 million.

profile approximately 800 symbols presented in MILC’s binary. We also set the overhead limit as 2s for each function. For this comparative analysis, we selected MAP, a prevalent sampling profiler, and TAU, a prevalent instrumented profiler, as the reference benchmarks. Table 3 lists all functions with an inclusive time that surpasses 3% of the runtime.

Table 3: Inclusive time for MILC

Function	MAP	TAU	PEAK
main	100% (463.20s)	100% (472.56s)	100% (477.41s)
update	98.60%	97.70%	97.08%
update_h_fermion	95.00%	93.39%	92.90%
eo_fermion_force_multi	67.20%	67.59%	66.20%
fn_fermion_force_multi_hisq_wrapper_mx_cpu	67.20%	67.57%	66.17%
fn_fermion_force_multi_hisq_smearing	66.20%	65.51%	63.68%
link_transport_connection_hisq	45.10%	41.66%	40.26%
ks_ratinv	25.20%	23.21%	23.34%
ks_multicg_field	25.20%	23.20%	23.33%
ks_congrad_parity_qphix	15.80%	15.31%	16.08%
ks_congrad_parity_qphix_F	15.80%	NA†	16.08%
mult_su3_an	15.20%	NA†	0.01%§
wait_gather	11.20%	11.62%	5.73%§
qphix_ks_dslash_F	10.10%	NA†	8.16%
ks_multicg_offset_field_qphix	10.40%	8.90%	8.16%
ks_multicg_offset_field_qphix_F	10.40%	NA†	8.16%
QPHIX_F3_asqtad_invert_multi	8.80%	NA†	6.59%
link_gather_connection_hisq	7.40%	9.29%	8.58%
QPHIX_F3_asqtad_create_L_from_raw	5.70%	NA†	6.05%
start_gather_field	5.10%	NA†	4.04%§
create_hisq_links_milc	4.30%	4.20%	4.92%
restore_fermion_links_from_site	4.10%	4.05%	4.48%
restore_fermion_links_hisq	4.10%	4.04%	4.47%
QPHIX_F3_asqtad_invert	4.00%	NA†	3.53%
load_fatlinks_cpu	3.90%	NA†	3.55%
compute_gen_staple_field	3.90%	3.32%	3.39%

† Not able to instrument some of the dependent lib with TAU.

§ Profiler is detached for these functions.

We observe that PEAK’s results align with those of the other two profilers, with the exception of measurements from detached functions. Notably, PEAK’s outcomes are more closely aligned with TAU’s than with MAP’s – a testament to the fundamental disparity between the two profiling approaches. It’s important to mention that TAU exclusively instruments code that has been recompiled with it, leading to some functions within dependent libraries remaining unprofiled in this run. The relatively higher overhead in PEAK compared to TAU can be attributed to the additional instrumentation of functions within dependent libraries. Regardless of the difference, all three profilers effectively capture the accurate performance characteristics of the application.

5 CONCLUSION

We introduce PEAK as a light-weight profiling solution tailored for large HPC applications. Our initial implementation showcases notably small overhead in comparison to established tools. Furthermore, the results show sufficient accuracy, particularly when the detachment mechanism is activated. Importantly, these accomplishments are realized without necessitating any modifications or recompilation of the application’s source code. The ongoing development endeavors aim to enhance the tool’s capabilities, with a view towards transforming it into a more robust and potent profiler.

ACKNOWLEDGMENTS

This work is supported by the National Science Foundation through award OAC-1854828. This work is also supported in part by the award OAC-2139536 and OAC-2103494.

REFERENCES

- [1] L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G. Marin, J. Mellor-Crummey, and N. R. Tallent. 2010. HPC-TOOLKIT: tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience* 22, 6 (2010), 685–701. <https://doi.org/10.1002/cpe.1553> _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.1553>.
- [2] Andrew R. Bernat and Barton P. Miller. 2011. Anywhere, any-time binary instrumentation. In *Proceedings of the 10th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools* (2011-09-05). ACM, Szeged Hungary, 9–16. <https://doi.org/10.1145/2024569.2024572>
- [3] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. 2003. An infrastructure for adaptive dynamic optimization. In *Proceedings of the international symposium on Code generation and optimization: feedback-directed and runtime optimization* (2003-03-23) (CGO ’03). IEEE Computer Society, USA, 265–275.
- [4] Galen Hunt and Doug Brubacher. 1999. Detours: Binary Interception of Win32 Functions. In *Proceedings of the 3rd Conference on USENIX Windows NT Symposium - Volume 3* (Seattle, Washington) (WINSYM’99). USENIX Association, USA, 14.
- [5] Christopher January, Jonathan Byrd, Xavier Oró, and Mark O’Connor. 2015. Allinea MAP: Adding Energy and OpenMP Profiling Without Increasing Overhead. In *Tools for High Performance Computing 2014*, Christoph Niethammer, José Gracia, Andreas Knüpfer, Michael M. Resch, and Wolfgang E. Nagel (Eds.). Springer International Publishing, Cham, 25–35. https://doi.org/10.1007/978-3-319-16012-2_2
- [6] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. *ACM SIGPLAN Notices* 40, 6 (2005), 190–200. <https://doi.org/10.1145/1064978.1065034>
- [7] Dieter An Mey, Scott Biersdorf, Christian Bischof, Kai Diethelm, Dominic Eschweiler, Michael Gerndt, Andreas Knüpfer, Daniel Lorenz, Allen Malony, Wolfgang E. Nagel, Yuri Oleyunik, Christian Rössel, Pavel Saviankou, Dirk Schmidt, Sameer Shende, Michael Wagner, Bert Wesarg, and Felix Wolf. 2011. Score-P: A Unified Performance Measurement System for Petascale Applications. In *Competence in High Performance Computing 2010*, Christian Bischof, Heinz-Gerd Hegering, Wolfgang E. Nagel, and Gabriel Wittum (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 85–97. https://doi.org/10.1007/978-3-642-24025-6_8
- [8] Ole André V Ravnäs. 2019. Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers.
- [9] James Reinders. 2005. *VTune performance analyzer essentials: measurement and tuning techniques for software developers* (1. print ed.). Intel Press, Hillsboro, Or.
- [10] Sameer S. Shende and Allen D. Malony. 2006. The Tau Parallel Performance System. *The International Journal of High Performance Computing Applications* 20, 2 (May 2006), 287–311. <https://doi.org/10.1177/1094342006064482> Publisher: SAGE Publications Ltd STM.
- [11] Dan Stanzione, John West, R. Todd Evans, Tommy Minyard, Omar Ghattas, and Dhabaleswar K. Panda. 2020. Frontera: The Evolution of Leadership Computing at the National Science Foundation. In *Practice and Experience in Advanced Research Computing* (Portland, OR, USA) (PEARC ’20). Association for Computing Machinery, New York, NY, USA, 106–111. <https://doi.org/10.1145/3311790.3396656>
- [12] Jeffrey Vetter and Chris Chabreanu. 2005. *mpiP: Lightweight, Scalable MPI Profiling*.

Received 16 August 2023; revised 8 September 2023; accepted 8 September 2023